Enter StringIO objects. They behave just like other I/O objects, but they read and write strings, not files. If you open a StringIO object for reading, you supply it with a string. All read operations on the StringIO object then read from this string. Similarly, when you want to write to a StringIO object, you pass it a string to be filled.

```
require 'stringio'

ip = StringIO.new("now is\nthe time\nto learn\nRuby!")
op = StringIO.new("", "w")

ip.each_line do |line|
  op.puts line.reverse
end
op.string # => "\nsi won\n\nemit eht\n\nnrael ot\n!ybuR\n"
```

Talking to Networks

Ruby is fluent in most of the Internet's protocols, both low-level and high-level.

For those who enjoy groveling around at the network level, Ruby comes with a set of classes in the socket library (https://docs.ruby-lang.org/en/master/Socket.html). These give you access to TCP, UDP, SOCKS, and Unix domain sockets, as well as any additional socket types supported on your architecture. The library also provides helper classes to make writing servers easier. Here's a simple program that gets information about our user website on a local web server using the HTTP OPTIONS request:

tut_io/socket.rb

```
require "socket"

client = TCPSocket.open("127.0.0.1", "www")
client.send("OPTIONS /~dave/ HTTP/1.0\n\n", 0)    # 0 means standard packet
puts client.readlines
client.close
```

At a higher level, the "lib/net" set of library modules provides handlers for a set of application-level protocols (currently FTP, HTTP, HTTPS, IMAP, POP, and SMTP). For example, the following program lists the images that are displayed on this book's home page. (To save space, we show only the first three):

tut_io/networking.rb

```
require "net/http"

uri = URI("https://pragprog.com/titles/ruby5/programming-ruby-3-2-5th-edition/")
Net::HTTP.start(
  "pragprog.com",
  Net::HTTP.https_default_port,
  use_ssl: true
) do |http|
  request = Net::HTTP::Get.new(uri)
  response = http.request(request)
  if response.code == "200"
    puts response.body.scan(/<img class=".*?" src="(.*?)"/m).uniq[0, 3]
  end
end
```

produces:

```
/titles/ruby5/programming-ruby-3-2-5th-edition/ruby5-beta-250.jpg
```